Project - High Level Design

on

Dockerized Sports AI Service

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	AANCHAL MARATHE	EN22CS301015
02	AAHMAD BURHANI	EN22CS301010
03	ABHISHEK SANKHALA	EN22CS301035
04	YASHASVI MODI	EN22CS3011114
05	AKSHAT YADAV	EN22CS301095

Group Name: 10D1

Project Number: AAI-10

Industry Mentor Name:

University Mentor Name: Dr. Ritesh Joshi

Academic Year: 2025-2026

Table of Contents

1. Introduction.
    1.1. Scope of the document.
    1.2. Intended Audience
    1.3. System overview.
2. System Design.
    2.1. Application Design
    2.2. Process Flow.
    2.3. Information Flow.
    2.4. Components Design
    2.5. Key Design Considerations
    2.6. API Catalogue.
3. Data Design
    3.1. Data Model
    3.2. Data Access Mechanism
    3.3. Data Retention Policies
    3.4. Data Migration
4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
    7.1 Security Aspects
    7.2 Performance Aspects
8. References

1. Introduction

This document provides a detailed Low-Level Design (LLD) of the **Dockerized Sports AI Service**, an intelligent sports planning assistant built using Python-based AI frameworks and deployed using modern DevOps practices.
The system leverages **agentic AI architecture** to process user queries related to sports data and planning. It integrates external APIs for real-time sports information and uses containerization technologies to ensure portability, scalability, and consistency across environments.
The application is deployed using Docker and orchestrated using Kubernetes on cloud infrastructure, ensuring high availability and efficient resource utilization.

1.1 Scope of the Document

The scope of this document includes the detailed design and implementation aspects of the system, covering:

- Internal architecture of the application components
- Interaction between frontend, backend, and AI agents
- API integrations and data flow
- Docker containerization strategy
- Kubernetes deployment configuration
- Environment configuration and dependency management

This document focuses on the **technical implementation details** and does not include high-level business requirements or project planning aspects.

1.2 Intended Audience

This document is intended for:

- **Developers** – to understand the system architecture and code-level design
- **DevOps Engineers** – to manage containerization, deployment, and scaling
- **Project Evaluators / Faculty** – to assess technical implementation and design quality
- **System Maintainers** – for troubleshooting and future enhancements

Basic knowledge of the following is recommended:

- Python programming
- Docker and containerization
- Kubernetes fundamentals
- REST APIs and web applications
- AI frameworks such as CrewAI and LangChain

1.3 System Overview

The system is a **Sports AI Assistant Application** that allows users to interact through a web interface and receive intelligent responses based on sports data.

Key Features:

- AI-powered query handling using agent-based reasoning
- Integration with external sports APIs for real-time data
- Interactive frontend built using Streamlit
- Secure handling of API keys and environment variables
- Containerized deployment using Docker
- Scalable orchestration using Kubernetes

High-Level Workflow:

1. User submits a query through the web interface
2. The request is processed by AI agents
3. Agents fetch data from external APIs if required
4. The system generates a contextual response using LLM
5. Response is displayed back to the user

Deployment Overview:

- Application is packaged as a Docker image
- Hosted on cloud infrastructure via Amazon Web Services
- Managed and scaled using Kubernetes

2. System Workflow:

Step 1: User Interaction

- User opens the web application (Streamlit UI)
- Enters a query like:
  - "Show upcoming matches"
  - "Analyze team performance"

Step 2: Request Submission

- The query is submitted via Streamlit frontend
- Sent to backend Python application

Step 3: Agent Initialization (CrewAI)

- CrewAI agent receives the query
- Agent determines:
  - Intent of the query
  - Required tools (API / reasoning)

Step 4: Reasoning using LangChain (ReAct)

- LangChain processes the request
- Applies **ReAct reasoning**:
  - Think → Decide → Act
- Determines whether external data is required

Step 5: External API Call

- If data is needed:
  - Request sent to The SportsDB API
- Data retrieved in JSON format

Step 6: Data Processing

- API response is cleaned and structured
- Passed to OpenAI LLM

Step 7: LLM Response Generation

- OpenAI model:
  - Interprets data
  - Generates human-like response
- Combines reasoning + real data

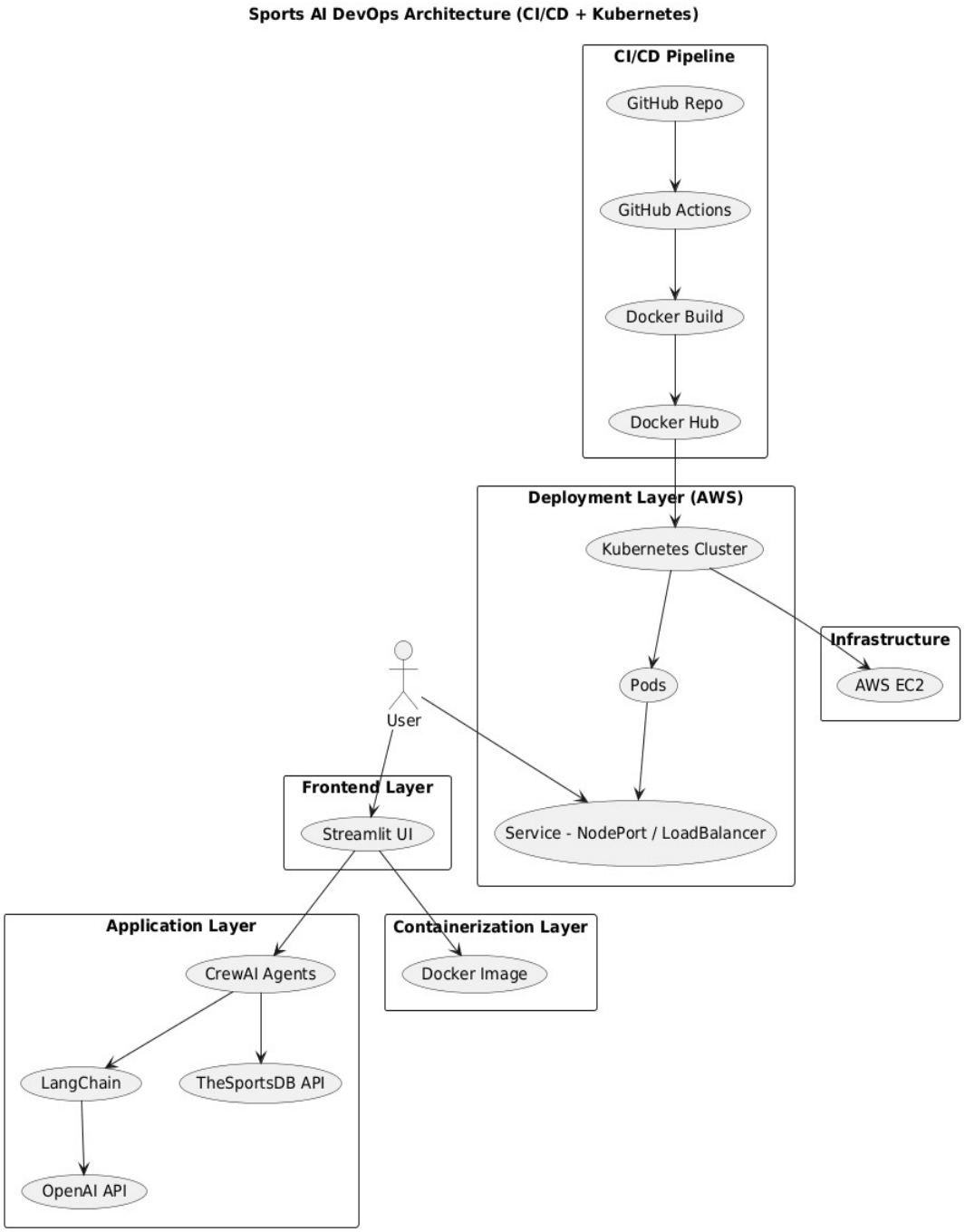
Step 8: Response Delivery

- Final response sent back to Streamlit UI

- Displayed to user in readable format

Step 9: Container Execution (DevOps Layer)

- Entire application runs inside Docker container
- Container managed by Kubernetes:
  - Handles scaling
  - Ensures high availability



2.1Application Design

The application is built using a **three-layer architecture**:

1. Presentation Layer

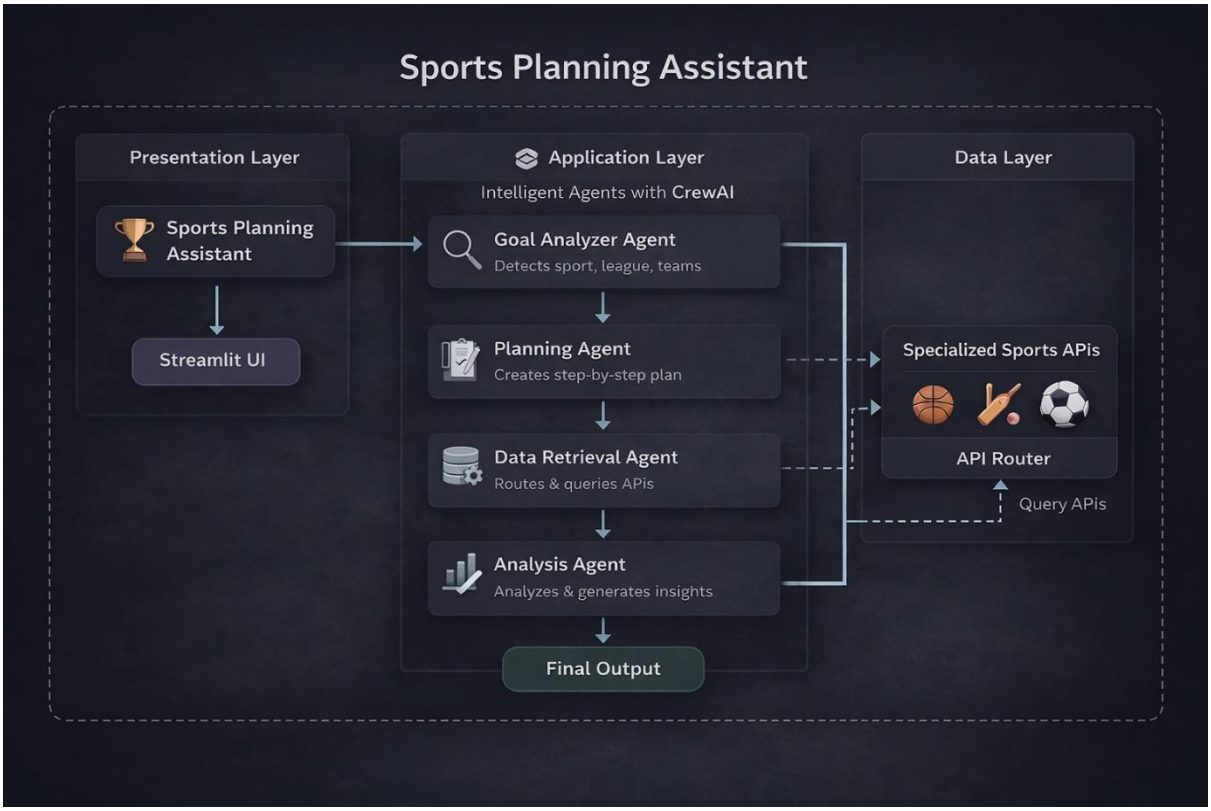
- Built using Streamlit
- Handles user input and displays AI-generated responses
- Provides an interactive UI for sports-related queries

2. Application (Logic) Layer

- Implements AI-based processing using:
  - CrewAI
  - LangChain
- Handles:
  - Query interpretation
  - Agent decision-making
  - Tool/API invocation

3. Data Layer

- Integrates external APIs like TheSportsDB API
- Retrieves real-time sports data
- Manages environment variables and API keys securely

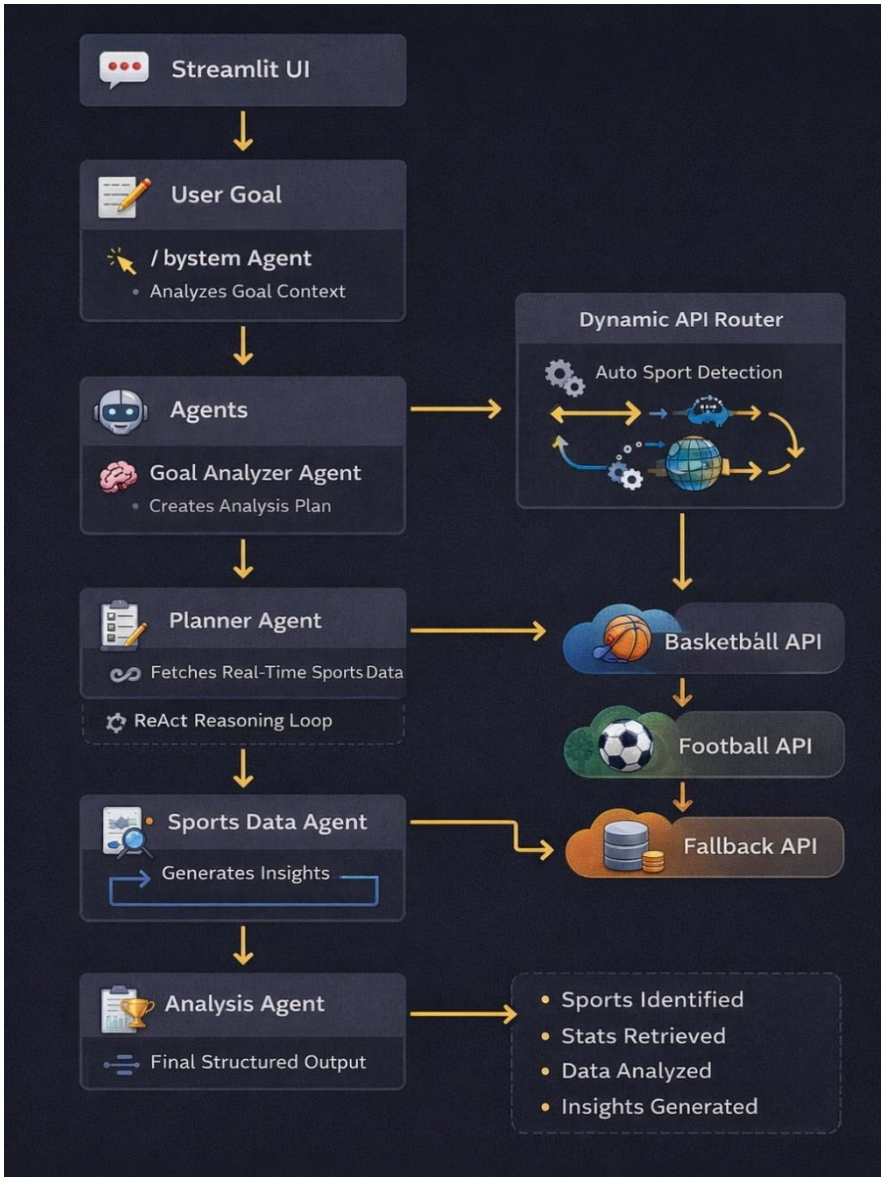


2.2 Process Flow

The process flow describes how a user request is handled:

1. User enters a query in the UI
2. Request is sent to the backend logic
3. AI agent interprets the query
4. If needed, external API is called
5. Data is processed using LLM
6. Response is generated
7. Output is displayed on UI

This flow ensures dynamic and intelligent responses using agent-based reasoning.



2.3 Information flow

The information flow defines how data moves across the system:

User → Streamlit UI → AI Agent → External API → LLM → Response → UI

Flow Description:

- Input: User query
- Processing: AI agents + LLM
- External Data: Sports API response
- Output: Contextual response

This ensures seamless integration between user interaction and intelligent processing.

2.4 Components Design

The system is divided into the following core components:

1. Frontend Component

- Built using Streamlit
- Handles user interaction

2. AI Processing Component

- CrewAI agents handle reasoning and decision-making
- LangChain manages LLM workflows

3. API Integration Component

- Communicates with TheSportsDB API
- Fetches real-time sports data

4. Configuration Component

- Uses .env file for managing secrets
- Ensures secure API key handling

5. Containerization Component

- Docker used for packaging the application
- Ensures consistency across environments

6. Orchestration Component

- Kubernetes manages deployment, scaling, and availability

2.5 Key Design Considerations

The following principles were considered while designing the system:

Scalability

- Kubernetes enables horizontal scaling

Portability

- Docker ensures application runs consistently across systems

Security

- API keys stored using environment variables
- Sensitive data not hardcoded

Modularity

- Separation of frontend, logic, and data layers

Performance

- Efficient API calls and lightweight containers

Maintainability

- Clear code structure and modular components

2.6 API Catalogue

The system uses the following APIs:

1. Sports Data API

- Name: TheSportsDB API
- Purpose: Fetch sports-related data (teams, matches, players)
- Method: GET
- Example Endpoint:

https://www.thesportsdb.com/api/v1/json/{API_KEY}/searchteams.php?t=Arsenal

2. LLM API

- Name: OpenAI API
- Purpose: Generate intelligent responses
- Method: POST
- Usage:

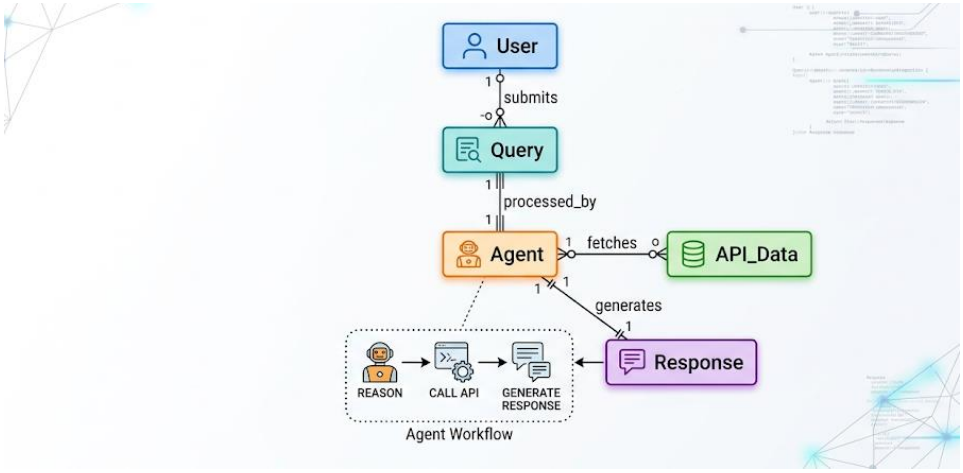
- Query understanding
- Response generation

3. Internal Agent Tools

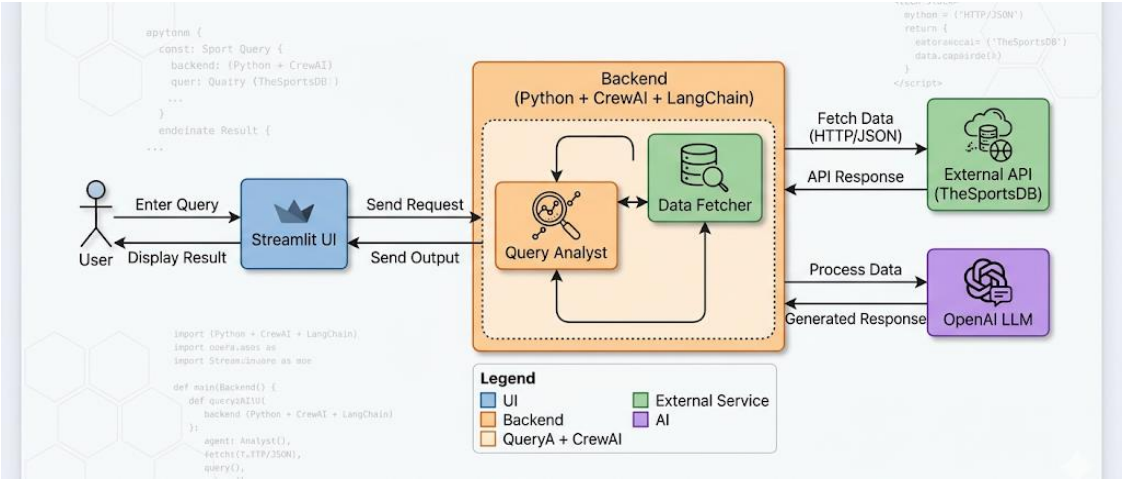
- Used within CrewAI framework
- Purpose:
  - Decision making
  - Tool calling
  - Task execution

3. Data Design

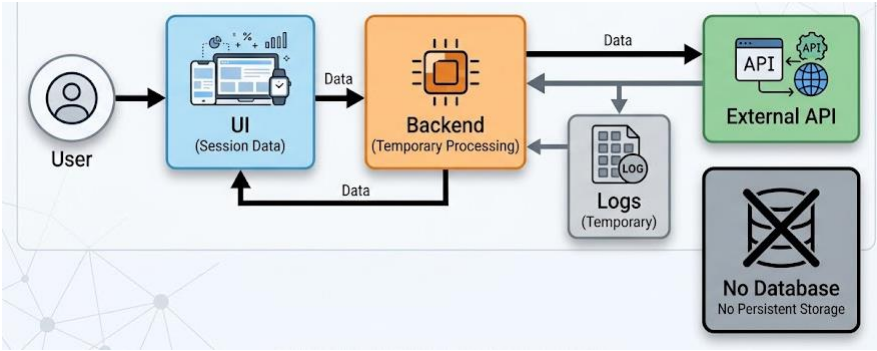
3.1 Data Model



3.2 Data Access Mechanism



3.3 Data Retention Policies



\

3.4 Data Migration

In the Dockerized AI Sports Assistant project, data migration is not required because the system is designed as a stateless application. It does not store any user data or application data in a database. Instead, all the required sports information is fetched dynamically in real-time from external APIs, and responses are generated using AI models such as CrewAI and LangChain. Since there is no persistent storage involved, there is no need to transfer or migrate data between systems. This design simplifies the architecture, reduces maintenance

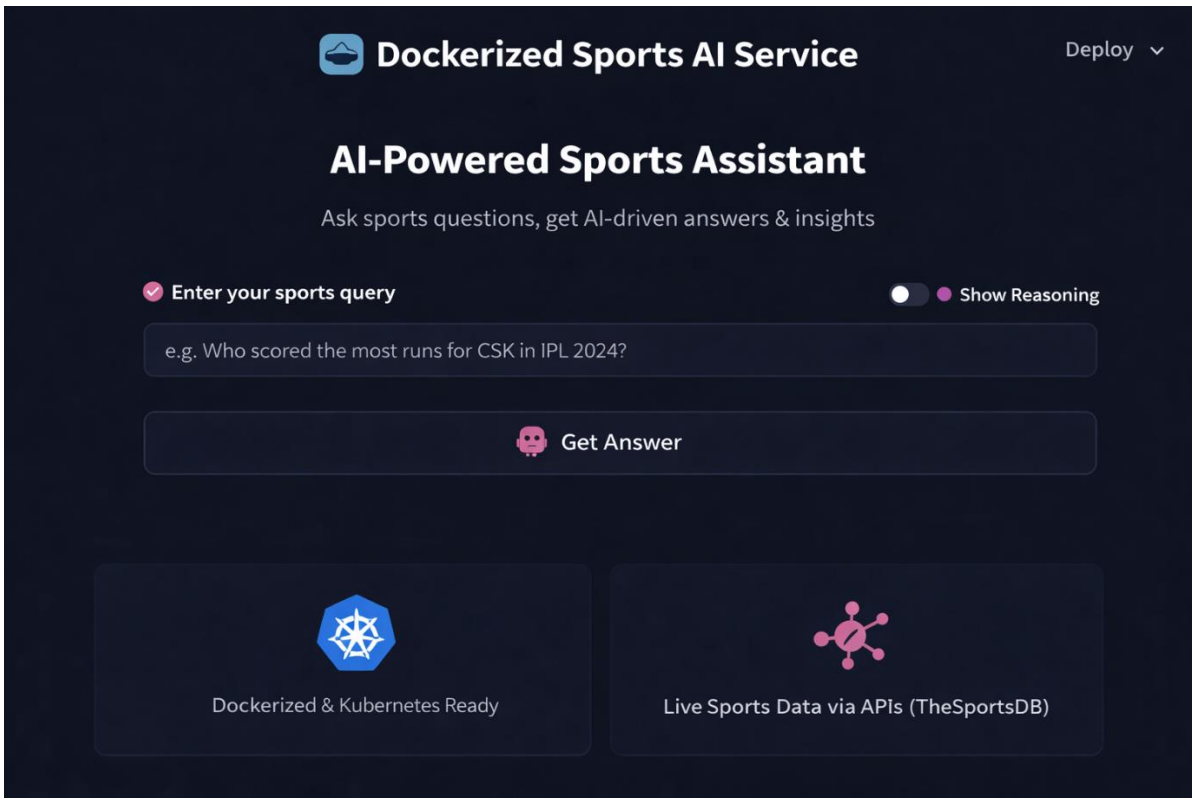
effort, and ensures that the system always provides up-to-date information without relying on stored data.

4. Interfaces

This section describes the interfaces between different system components including the **User Interface, LLM services, and Image Generation services**.

Types of Interfaces:

- User Interface (Web-based Frontend)
- Content Generation Interface



5. State and Session Management

In the Dockerized AI Sports Assistant system, state and session management are handled in a lightweight and stateless manner. The application does not maintain any persistent state on the server side. Each user request is processed independently without storing previous interactions. Session handling is managed at the frontend level using Streamlit, which temporarily maintains user inputs during a single interaction session. Once the session ends or the application is refreshed, the data is cleared automatically. This stateless approach improves scalability, simplifies deployment in Docker and Kubernetes environments, and ensures that multiple users can interact with the system concurrently without session conflicts.

processing user requests in real time.

6. Caching

Caching is an important non-functional aspect of the **Sports Planning Assistant Agent**, aimed at improving system performance, reducing API latency, and minimizing redundant computations. Since the system relies on external APIs and multi-step agent reasoning, caching helps in optimizing response time and resource usage.

Caching Strategy

The system primarily uses **in-memory caching techniques** during runtime. Since the backend is designed to be lightweight and stateless, caching is implemented at the application level rather than using a dedicated caching server.

Caching can be applied at multiple stages:

1. API Response Caching

- Responses received from external APIs such as TheSportsDB API can be temporarily stored.
- If the same team, league, or player data is requested again, the system retrieves it from cache instead of making a new API call.
- This significantly reduces network overhead and latency.

2. Agent Output Caching

- Intermediate outputs generated by agents (e.g., detected sport, league, or team) can be cached during a session.

- This prevents repeated processing of the same input within the same workflow.

3. Visualization Data Caching

- Processed data used for generating charts and graphs can be cached to avoid recomputation when rendering visualizations multiple times.

Implementation Approach

- Caching is implemented using built-in mechanisms available in Streamlit such as:
  - st.cache_data() for storing API responses
  - st.session_state for maintaining temporary session data
- Cached data is stored in memory and is cleared automatically when:
  - The application restarts
  - The session expires

Cache Invalidation

To maintain data accuracy, the system follows simple cache invalidation strategies:

- Cached data is refreshed periodically to ensure up-to-date information
- In cases where real-time accuracy is critical, fresh API calls are preferred over cached data
- Cache is cleared when session ends to avoid stale data usage

7. Non-Functional Requirements

Non-functional requirements define the quality attributes ensuring that the system performs efficiently, securely, and reliably while handling user requests.

7.1 Security Aspects

- API keys stored in .env
- Kubernetes Secrets for production
- Secure container images
- Firewall rules in AWS

7.2 Performance Aspects

- Fast response via LLM optimization
- Container scaling via Kubernetes
- Load balancing using K8s Services

8. References

- Docker Documentation
- Kubernetes Documentation
- OpenAI API Docs
- CrewAI Docs
- LangChain Docs
- TheSportsDB API Docs